

基于 MongoDB 的问卷系统核心文档（第二阶段 · 需求变更版）

张子扬 10245102456

💡 提示：以下为精简过后的文档。完整的实验文档等信息，请查阅项目仓库中的 docs，内容更加详实。

1. 第一阶段设计回顾

1.1 原系统架构

第一阶段系统采用 Node.js + Express + MongoDB 技术栈，实现了完整的问卷管理流程。核心架构如下：

- **用户认证模块**：JWT Token 鉴权。
- **问卷管理模块**：问卷的创建、发布、关闭、删除。
- **题目管理模块**：题目的增删改，与问卷强绑定。
- **填写提交模块**：通过分享码获取问卷与题目，提交答案并完成跳转校验。
- **统计分析模块**：按问卷维度的整卷统计与单题统计。

1.2 原数据模型

第一阶段定义了 4 个核心集合：

集合	用途	核心关系
users	用户账号	—
surveys	问卷元信息	creatorId → users
questions	题目内容	surveyId → surveys
responses	提交回答	surveyId → surveys

关键设计特征：题目（Question）通过 `surveyId` 与问卷强绑定，每道题归属于且只归属于一个问卷。删除问卷时会级联删除所有关联题目。

1.3 原系统的局限

第一阶段的设计以“一对一”关系为核心——题目嵌入在特定问卷中，无法独立存在或复用。这在简单场景下简洁高效，但在实际使用中暴露出以下限制：

1. 每次创建问卷都需重新编写题目，无法复用。
2. 修改题目会直接影响所有引用处，包括已发布的问卷。
3. 题目无版本概念，无法回溯历史变更。
4. 统计只能按单个问卷维度查看，无法跨问卷分析。

2. 新需求分析与设计挑战

2.1 需求变更概述

根据用户反馈，第二阶段新增 8 项需求：

编号	需求	核心诉求
一	保存常用题目	题目独立存在，可多问卷复用
二	分享题目	题目可共享给其他用户
三	修改不影响已发布	已发布问卷的题目内容不可变
四	修改历史	题目版本链可追溯
五	多版本共存	不同问卷可使用同题不同版本
六	查看引用	可反查题目被哪些问卷使用
七	题目库	按分类管理常用题集合
八	跨问卷统计	同一题在所有问卷中的汇总统计

2.2 新需求带来的核心矛盾

上述需求的根本冲突在于第一阶段的**题目-问卷"一对一强绑定"模型**：

- 1. **复用矛盾**： `surveyId` 将题目锁定在单个问卷内，无法被多个问卷引用。
- 2. **不可变矛盾**： 题目直接修改会影响所有使用处，与"已发布不可变"需求冲突。
- 3. **统计矛盾**： 统计以 `surveyId` 为聚合维度，无法跨问卷聚合同一题目。

结论： 必须将题目从"问卷的附属品"重构为"独立实体"，同时引入版本管理机制。

3. 设计修改方案

3.1 修改原则

严格遵循"基于第一阶段修改"的要求：

- **保留**： 用户认证、问卷管理、填写提交、统计分析等全部原有功能。
- **保留**： Express 路由结构、Mongoose 模型层、JWT 鉴权中间件、跳转逻辑引擎、答案校验引擎。
- **修改**： Question 模型（从从属改为独立）、Survey 模型（增加引用方式）。
- **新增**： QuestionBank 模型、题目库/题库管理路由、跨问卷统计路由。

3.2 核心架构变化：题目独立化 + COW 版本管理

第一阶段：Survey —1:N—> Question (强绑定)
第二阶段：Survey —questionRefs—> Question (引用, 多对多)
 Question 自身形成版本链 (rootQuestionId → 版本树)

COW (Copy-on-Write) 策略：

- 当用户修改一个被已发布问卷引用的题目时，系统**不直接修改原题**，而是创建一份新版本。
- 旧版本保持不变，已发布问卷仍然引用旧版本。
- 草稿状态的问卷自动切换到新版本。

4. 数据结构变化

4.1 questions 集合（重构）

第一阶段的 questions 以 `surveyId + order` 定位题目。第二阶段移除这两个字段，改为独立实体。

移除的字段：

- `surveyId`：题目不再从属于特定问卷。
- `order`：题序改由 `survey.questionRefs` 管理。

新增的字段：

字段	类型	说明
<code>creatorId</code>	<code>ObjectId</code>	题目创建者，关联 <code>users._id</code>
<code>sharedWith</code>	<code>ObjectId[]</code>	被共享的用户 ID 列表
<code>rootQuestionId</code>	<code>ObjectId</code>	版本链根节点，首版指向自身
<code>parentVersionId</code>	<code>ObjectId</code>	上一版本的 <code>_id</code>
<code>version</code>	<code>Number</code>	版本号，从 1 开始递增
<code>isLatest</code>	<code>Boolean</code>	是否为最新版本
<code>createdAt</code>	<code>Date</code>	创建时间
<code>updatedAt</code>	<code>Date</code>	更新时间

保留的字段（完全不变）：

- `type`、`title`、`required`、`options`、`validation`、`jumpRules`

索引变更：

- 移除：`surveyId + order` 复合索引。
- 新增：`creatorId` 索引、`rootQuestionId` 索引、`sharedWith` 索引、`isLatest` 索引。

4.2 surveys 集合（扩展）

保留所有原字段，新增 `questionRefs` 数组用于引用题目：

```
questionRefs: [{
  questionId: ObjectId, // 引用特定版本的题目
  order: Number         // 题目在该问卷中的顺序
}]
```

设计意图：题序信息从 Question 移到 Survey 侧，因为同一题在不同问卷中可以有不同的顺序。

4.3 questionbanks 集合（新增）

字段	类型	说明
_id	ObjectId	主键
name	String	题库名称
description	String	题库说明
creatorId	ObjectId	创建者
questionIds	ObjectId[]	题目的 rootQuestionId 列表
createdAt / updatedAt	Date	时间戳

设计说明：题库存储的是 `rootQuestionId`（版本链根 ID），而非具体某个版本的 ID。当题目更新新版本时，题库自动跟踪最新版本，无需手动更新。

4.4 responses 集合（无变化）

responses 的结构完全保持不变。`answers.questionId` 自然指向特定版本的题目 ID，在题目版本化后仍然有效——每个版本有自己的 `_id`，旧 response 引用的旧版本 ID 始终有效。

4.5 数据关系变化对比

第一阶段：

User —1:N—> Survey —1:N—> Question

Survey —1:N—> Response

第二阶段：

User —1:N—> Survey —questionRefs—> Question（多对多）

User —1:N—> Question（独立拥有）

User —1:N—> QuestionBank —questionIds—> Question

Question —rootQuestionId—> Question（版本链，自关联）

Survey —1:N—> Response

5. 程序调整说明

5.1 后端路由调整

保留并修改的路由

路由文件	修改内容
surveys.js	发布检查改为 <code>questionRefs.length</code> ；删除时不再级联删 Question
fill.js	获取题目改为从 <code>survey.questionRefs</code> 加载而非通过 <code>surveyId</code> 查询

路由文件	修改内容
stats.js	统计时从 <code>questionRefs</code> 获取题目 ID 列表
questions.js	重写为引用管理：从题目库选题、问卷内新建题目、调序、移除引用

新增的路由

路由文件	路径前缀	功能
questionLib.js	/api/questions	独立题目 CRUD、共享、版本管理、引用查看、跨问卷统计
questionBank.js	/api/question-banks	题库 CRUD、题目添加/移除

路由优先级

在 `app.js` 中，`/api/questions` (`questionLib`) 必须注册在 `/api` (`questions`) 之前，以确保 `PUT /api/questions/:id` 由独立题目管理路由处理，而非问卷内兼容路由。

5.2 题目管理方式变化

第一阶段的题目操作方式：

```
POST /api/surveys/:surveyId/questions → 直接在问卷内创建题目
```

第二阶段的两种方式并存：

```
方式一：POST /api/surveys/:surveyId/questions/new → 问卷内新建（同时入题目库）
方式二：POST /api/surveys/:surveyId/questions → 从题目库选取已有题目
```

5.3 版本管理逻辑

更新题目时的分支判断：

- 1. 若该题未被任何已发布问卷引用 → 直接修改原题。
- 2. 若被已发布问卷引用 → 执行 COW：
 - 将原题 `isLatest` 设为 `false`。
 - 创建新文档，`version + 1`，`parentVersionId` 指向旧版。
 - 自动更新所有草稿问卷中的引用到新版本。
 - 已发布问卷的引用保持不变（指向旧版本）。

版本恢复：恢复旧版本时不是回退，而是以旧版本内容创建新版本（版本号继续递增），保证版本链单向增长。

5.4 跨问卷统计逻辑

跨问卷统计（`GET /api/questions/:id/stats`）的聚合流程：

- 1. 根据输入 ID 找到 `rootQuestionId`，获取该题所有版本的 ID 列表。
- 2. 查找所有 `questionRefs` 中包含这些 ID 的问卷。
- 3. 获取这些问卷的全部 Response。
- 4. 从 Response 中提取匹配的答案进行整体聚合。
- 5. 同时按问卷分组输出每个问卷的回答数、占比及各选项分布。

5.5 保留不变的逻辑

以下模块保持与第一阶段完全一致，未做任何修改：

- **JWT 鉴权与可选鉴权**：auth.js 中间件不变。
- **答案校验引擎**：validation.js 不变。
- **跳转逻辑引擎**：jumpLogic.js 不变。
- **Response 数据模型**：结构不变。
- **User 数据模型**：结构不变。

6. 兼容性问题与解决

6.1 题目归属方式变化

问题	解决方案
题目不再有 <code>surveyId</code> ，如何知道属于哪个问卷？	通过 <code>survey.questionRefs</code> 反查
题目不再有 <code>order</code> ，如何确定题序？	<code>order</code> 存储在 <code>survey.questionRefs[].order</code> 中
fill.js 原来通过 <code>surveyId</code> 查 Question，现在怎么办？	改为从 <code>survey.questionRefs</code> 加载题目 ID，再批量查询

6.2 删除行为变化

第一阶段	第二阶段	原因
删除问卷 → 级联删 Question + Response	删除问卷 → 只删 Response	题目现在是独立实体，可能被其他问卷引用
删除题目 → 直接删	删除题目 → 检查是否被引用，有引用则拒绝	防止破坏其他问卷

6.3 版本化与旧数据兼容

`rootQuestionId` 字段设计为：首次保存时若为空则自动设为自身 `_id`（通过 post-save hook 实现）。这确保：

- 单版本题目的 `rootQuestionId === _id`，查版本历史只返回自身。
- 多版本题目通过同一个 `rootQuestionId` 串联。

6.4 前端兼容

- dashboard.html 增加题目库/题库导航入口，原有问卷管理功能不受影响。

- create.html 增加"从题目库选题"入口，原有"问卷内新建"方式保留。
- fill.html 和 stats.html 的前端逻辑不变，后端接口返回格式保持一致。

7. 旧功能可用性验证

通过 23 个自动化 E2E 测试验证所有功能：

原有功能回归（12 项全部通过）

1. 创建问卷成功/失败：2/2 通过
2. 添加四类题型：通过（改为新模式后增加到 3/3）
3. 跳转逻辑-命中/未命中：2/2 通过
4. 校验-必答/规则：2/2 通过
5. 提交-匿名/非匿名：2/2 通过
6. 统计-整卷/权限：2/2 通过

新增功能测试（11 项全部通过）

7. 题目独立管理（创建/多问卷使用）：2/2 通过
8. 题目共享：1/1 通过
9. 版本管理（修改版本/查看历史/恢复旧版）：3/3 通过
10. 修改不影响已发布问卷：1/1 通过
11. 查看题目引用：1/1 通过
12. 题库管理：1/1 通过
13. 跨问卷统计：1/1 通过

8. API 接口补充说明

8.1 独立题目管理接口（需登录）

1. POST /api/questions — 创建独立题目
2. GET /api/questions — 获取个人题目列表（含被共享的，默认只显示最新版本）
3. GET /api/questions/:id — 获取题目详情
4. PUT /api/questions/:id — 更新题目（自动判断是否创建新版本）
5. DELETE /api/questions/:id — 删除题目（有引用时拒绝）
6. POST /api/questions/:id/share — 共享题目给指定用户
7. DELETE /api/questions/:id/share/:userId — 取消共享
8. GET /api/questions/:id/versions — 获取版本历史
9. POST /api/questions/:id/revert/:versionId — 恢复旧版本
10. GET /api/questions/:id/surveys — 查看题目被哪些问卷使用
11. GET /api/questions/:id/stats — 跨问卷统计（含按问卷分组明细）

8.2 问卷内题目管理接口（需登录，需草稿状态）

1. POST /api/surveys/:surveyId/questions — 从题目库选题添加
2. POST /api/surveys/:surveyId/questions/new — 在问卷内直接新建题目
3. GET /api/surveys/:surveyId/questions — 获取问卷题目列表（按 order 排序）

- 4. PUT /api/surveys/:surveyId/questions/reorder — 调整顺序
- 5. DELETE /api/surveys/:surveyId/questions/:questionId — 从问卷移除引用

8.3 题库管理接口（需登录）

- 1. POST /api/question-banks — 创建题库
- 2. GET /api/question-banks — 获取我的题库列表
- 3. GET /api/question-banks/:id — 获取题库详情（含题目列表）
- 4. PUT /api/question-banks/:id — 更新题库信息
- 5. DELETE /api/question-banks/:id — 删除题库
- 6. POST /api/question-banks/:id/questions — 向题库添加题目
- 7. DELETE /api/question-banks/:id/questions/:questionId — 从题库移除题目

9. 文件变更清单

后端 Models (server/models/)

文件	操作	说明
Question.js	重构	移除 surveyId/order; 新增 creatorId/sharedWith/版本链字段
Survey.js	扩展	新增 questionRefs 数组
QuestionBank.js	新增	题库模型
Response.js	不变	—
User.js	不变	—

后端 Routes (server/routes/)

文件	操作	说明
questionLib.js	新增	独立题目管理全部接口
questionBank.js	新增	题库 CRUD 接口
questions.js	重写	改为引用管理方式
fill.js	修改	适配 questionRefs 加载题目
stats.js	修改	适配 questionRefs 加载题目
surveys.js	修改	发布检查改为 questionRefs、删除不级联删 Question
app.js	修改	注册新路由、调整优先级
auth.js	不变	—

后端 Utils (server/utils/)

文件	操作	说明
----	----	----

文件	操作	说明
validation.js	不变	答案校验引擎完全保留
jumpLogic.js	不变	跳转逻辑引擎完全保留

前端 (public/)

文件	操作	说明
question-lib.html + js	新增	题目库管理页
question-bank.html + js	新增	题库管理页
dashboard.html	修改	导航栏增加入口
create.html + js	修改	增加从题目库选题功能
fill.html + js	不变	—
stats.html + js	不变	—

10. 小结

10.1 设计演进逻辑

第一阶段的"题目嵌入问卷"模型简洁但不可复用。第二阶段通过将题目独立化 + COW 版本链的设计，在保留原有全部功能的基础上，实现了题目复用、版本管理、共享协作和跨问卷统计四大能力。

10.2 修改策略

- **最小化侵入**：保留 auth、validation、jumpLogic 等核心引擎不变。
- **渐进式扩展**：在 Survey 上增加 questionRefs 而非替换整个模型。
- **向前兼容**：Response 结构不变，已有数据自然兼容。

10.3 核心设计原则（延续第一阶段）

本系统继续以"后端规则权威化"为核心：

- 前端负责交互体验。
- 后端负责权限、状态、校验、跳转、版本管理与统计。
- COW 版本控制在后端自动执行，前端无需感知版本链细节。

11. AI 工具使用说明

💡 提示：项目仓库 docs/AI说明.md 中包含完整的、带有具体 Prompt 和决策推演过程的详细 AI 使用报告，请老师务必查阅该文档以获取更全面的信息。以下仅为摘要。

11.1 使用的 AI 工具

工具	用途
----	----

工具	用途
Gemini 3.1 Pro	架构方案讨论、数据模型设计推演、版本管理策略对比
Claude Opus	代码实现、路由重构、测试用例编写、文档撰写
Cursor IDE 内置 AI	代码补全、Lint 修复、跨文件引用检查

11.2 AI 参与的关键决策节点

(一) 题目独立化方案选型

AI 给出三种方案（快照复制 / 引用+COW / 完全解耦+中间表），我基于数据冗余、跨问卷统计可行性和 MongoDB 查询特性，最终选定**引用+COW** 方案。详细的方案对比与决策理由见 docs/AI说明.md 第二章。

(二) 版本链字段设计

关于 rootQuestionId 与 parentVersionId 是否需要同时存在、isLatest 是否冗余，我与 AI 进行了多轮讨论。最终采用混合方案——rootQuestionId 支撑高效的版本族查询，parentVersionId 保留派生关系，isLatest 作为列表页查询的性能优化字段。

(三) 共享机制设计

AI 提出 permissions 独立集合方案（灵活但过度设计）和 sharedWith 数组方案（简洁但有规模上限）。基于 YAGNI 原则和当前使用场景，我选择了 sharedWith 数组。

(四) 跨问卷统计聚合策略

AI 建议使用 MongoDB Aggregation Pipeline，我评估后选择应用层聚合——代码可读性更好，且在课程项目数据量级下 Pipeline 性能优势不显著。

11.3 AI 生成代码的人工修正案例

路由优先级冲突：AI 在 app.js 中将 /api (questions.js) 注册在 /api/questions (questionLib.js) 之前，导致独立题目的 PUT /api/questions/:id 路由永远被问卷内的兼容路由拦截。我通过分析测试失败日志定位问题，手动调整注册顺序后 3 个失败的版本管理测试全部通过。

这是"AI 不感知 Express 中间件 first-match-wins 特性"的典型案列，说明全局一致性判断仍依赖开发者。

11.4 AI 协作方法论

阶段	策略
需求分析	约束前置——先列硬约束再让 AI 发散
方案设计	多方案对比——要求 AI 给出 ≥2 种方案 + trade-off
代码实现	分模块迭代——每次只处理一个路由/页面，便于审查
测试验证	跑测试 + 读日志——不盲信 AI 的"应该能跑"
文档撰写	AI 起稿 + 人工校准——AI 写结构和表格，我补充判断与决策

11.5 总结

AI 是高效的“技术顾问 + 代码工人”，但架构决策、约束判断和全局一致性仍然是开发者不可让渡的核心能力。本项目中 AI 节省了大量模板代码编写和方案调研时间，但每个关键分歧点的最终决策均由我做出，且所有 AI 生成代码均经过自动化测试验证。

⚠ 仓库地址（请老师务必留意）

本文档为精简版核心报告。 项目仓库中包含完整的源代码、详细的 AI 使用报告、逐项测试样例说明（含截图）、项目启动指南等，内容远比本文档详实：

🔗 <https://github.com/Kevin23-design/DataManagementSystem-26sp>

仓库 `big_project_1_2` 目录结构：

- `public/`：前端静态页面与样式、脚本文件。
- `server/`：后端服务代码（Express + MongoDB），包含路由、模型、中间件与工具函数。
- `docs/`：项目文档，包括：
 - `核心文档.md` — 本文档完整版
 - `AI说明.md` — 详细的 AI 使用过程（含 Prompt、方案对比、决策理由）
 - `测试样例说明.md` — 23 项测试用例说明 + 手工测试截图
 - `项目启动说明.md` — 环境准备与运行指南